

Inigo Quilez :: articles :: simple pathtracing - 2012



Intro

Writing a global illumination renderer takes anything between one hour and one weekend. Starting from scratch, I promise. But writing an efficient and general production ready global illumination renderer takes from one year to one decade.

When doing computer graphics as an aficionado rather than a professional, the "efficient" and "general" aspect can be dropped from your implementations. Which means you can indeed write a full global illumination renderer in one hour. Also, given the power of the hardware these days, even if you don't do any clever optimizations or algorithms, a global illumination system can render in a matter of seconds or even realtime.



A path traced fractal, brute forced, rendered in around one minute

For those of us who wasted hours and hours waiting to get a simple 2D low resolution basic fractal rendered back in the early 90s, today's brute-force raw power of machines seems pretty astonishing. In my opinion, the main advantage of fast hardware is not that the graphics get rendered quicker, but that clever algorithms are not that necessary anymore, meaning that straight away approaches (those which are actually the most

intuitive of all) can be directly coded and a result can be expected in a reasonable amount of time. 20 years ago expensive techniques required the implementation of clever, complex and obscure algorithms, making the entry level for the computer graphics hobbyist much higher. But thanks to new hardware that's not true anymore - today, **writing a global illumination renderer takes one hour** at most.

What you need first

So, let's say you have been doing some ray-based rendering lately and that you have the following functions available to you:

```
vec2 worldIntersect( in vec3 ro, in vec3 rd, in float maxlen );  
float worldShadow(   in vec3 ro, in vec3 rd, in float maxlen );
```

which compute the intersection of a ray with the geometry of a 3D scene. The worldIntersect function returns the closest intersection of ray with origin *ro* and normalized direction *rd* in the form of a distance and an object ID. In the other hand, worldShadow returns the existence of any intersection (or well, it returns 1.0 if there isn't any intersection happening and 0.0 if there is any intersection). The implementation of these functions depends on the context of your application. If you are **raytracing hand modeled objects**, these functions probably traverse a kd-tree/bih or a bvh (bounding volume hierarchy). If you are rendering procedural models, you are perhaps implementing these two functions as **raymarching in a distance field**. If you are modelling **terrains** or **3d fractals** or **voxels** you most probably have specialized intersection functions.

```
vec3 worldGetNormal( in vec3 po, in float objectID );  
vec3 worldGetColor( in vec3 po, in vec3 no, in float objectID );  
vec3 worldGetBackground( in vec3 rd );
```

The first two functions return the normal and surface color at a given surface point in the 3D scene, and the third returns a background/sky color so we can return a color for primary rays that don't hit any geometry.

```
void worldMoveObjects( in float ctime );  
mat4x3 worldMoveCamera( in float ctime );
```

These two functions move the object in the scene and position the camera for a given animation time.

```
vec3 worldApplyLighting( in vec3 pos, in vec3 nor );
```

This function computes the direct lighting a given point and a normal on the surface of the 3D scene. This is where regular point, directional, spot, dome or area lighting is done, and it includes the cast of shadow rays.

Once you have these functions, implementing a pathtracer for global illumination is not any more difficult than it is to implement a regular raytracing renderer.

A classic direct lighting renderer

If you are reading this article, it probably means you have implemented stuff like this already a million times:

```
//-----
// run for every pixel in the image
//-----
vec3 calcPixelColor( in vec2 pixel, in vec2 resolution, in float frameTime )
{
    // screen coords
    vec2 p = (2.0*pixel(-resolution) / resolution.y);

    // move objects
    worldMoveObjects( frameTime );

    // get camera position, and right/up/front axis
    vec3 (ro, uu, vv, ww) = worldMoveCamera( frameTime );

    // create ray
    vec3 rd = normalize( p.x*uu + p.y*vv + 2.5*ww );

    // calc pixel color
    vec3 col = rendererCalculateColor( ro, rd );

    // apply gamma correction
    col = pow( col, 0.45 );

    return col;
}
```

which is the main entry function that computes the color of every pixel of the image, followed by the function that initiates the actual ray casting process:

```
vec3 rendererCalculateColor( vec3 ro, vec3 rd )
{
    // intersect scene
    vec2 tres = worldIntersect( ro, rd, 1000.0 );

    // if nothing found, return background color
    if( tres.y < 0.0 )
        return worldGetBackground( rd );

    // get position and normal at the intersection point
    vec3 pos = ro + rd * tres.x;
    vec3 nor = worldGetNormal( pos, tres.y );
```

```

// get color for the surface
vec3 scol = worldGetColor( pos, nor, tres.y );

// compute direct lighting
vec3 dcol = worldApplyLighting( pos, nor );

// surface * lighting
vec3 tcol = scol * dcol;

return tcol;
}

```

This is indeed a regular direct lighting renderer, as used in most intros, demos and games.

Note that when rendering with rays, it all starts by iterating the pixels of the screen. So if you are writing a CPU tracer, you probably want to do this by splitting the screen in tiles of say, 32x32 pixels, and by consuming the tiles by a pool of threads that contain as many threads as cores you have. You can see code that does that [here](#). If you are in the GPU, like in a fragment shader or a compute shader, then that work is done for you. Either case, we have a function calcPixelColor() that needs to compute the color of a pixel given its coordinates in screen and a scene description (the scene description is given by the functions above).

The montecarlo path tracer

As said, the point of this article is to keep things simple and not be too smart. So we are writing our Montecarlo tracer in quite a brute force manner.

We of course start from the pixels, and the easiest way to get our rays randomized by blindly sampling the pixel area to get antialiasing, the lens of the camera to get depth of field, and the animation across the frame to get motion blur. For free. Since we will do this random sampling for every ray, then light integration and these other effects happen simultaneously, which is pretty nice. Imagine we were using 256 light paths/samples per pixel to get a good noise-free illumination. Then we would be effectively computing 256x antialiasing for free. Neat. So, the main rendering function that runs for every pixel looks something like this:

```

// compute the color of a pixel
vec3 calcPixelColor( in vec2 pixel, in vec2 resolution, in float frameTime )
{
    float shutterAperture = 0.6;
    float fov = 2.5;
    float focusDistance = 1.3;
    float blurAmount = 0.0015;
    int numLevels = 5;

```

```
// 256 paths per pixel
vec3 col = vec3(0.0);
for( int i=0; i<256; i++ )
{
    // screen coords with antialiasing
    vec2 p = (2.0*(pixel + random2f())-resolution) / resolution.y;

    // motion blur
    float ctime = frameTime + shutterAperture*(1.0/24.0)*random1f();

    // move objects
    worldMoveObjects( ctime );

    // get camera position, and right/up/front axis
    vec3 (ro, uu, vv, ww) = worldMoveCamera( ctime );

    // create ray with depth of field
    vec3 er = normalize( vec3( p.xy, fov ) );
    vec3 rd = er.x*uu + er.y*vv + er.z*ww;

    vec3 go = blurAmount*vec3( -1.0 + 2.0*random2f(), 0.0 );
    vec3 gd = normalize( er*focusDistance - go );
    ro += go.x*uu + go.y*vv;
    rd += gd.x*uu + gd.y*vv;

    // accumulate path
    col += rendererCalculateColor( ro, normalize(rd), numLevels );
}
col = col / 256.0;

// apply gamma correction
col = pow( col, 0.45 );

return col;
}
```



A frame with depth of field, motion blur and 256x antialiasing, rendered with the code above this image

Note that the `worldMoveObjects()` and `worldMoveCamera()` function will position all the objects in the scene and the camera for a given time passed as argument. Of course repositioning all the objects can be expensive in some contexts (not in procedurally defined scenes, but in BVH/KDTree based scenes), you might want to implement time jittering for motion blur differently, like passing the shutter time as part of the ray information and then linearly interpolating polygons positions based on that. But for simple procedural graphics, the approach above is just simple and easy :)

Another note difference is that now `rendererCalculateColor()` receives an integer with the amount of levels of recursive raytracing we will want for our tracer (which is one plus the amount of light bounces - but more to come on this topic soon).

The ball is now in `rendererCalculateColor()`'s roof. This function, given a ray and the scene, has to compute a color. As with the classic direct lighting renderer, we start by casting our ray against the scene geometry looking for an intersection, computing the position and normal at the intersection point, getting the local surface color of the object that was hit, and then computing local lighting.

```
vec3 rendererCalculateColor( vec3 ro, vec3 rd, int numLevels )
{
    // intersect scene
    vec2 tres = worldIntersect( ro, rd, 1000.0 );

    // if nothing found, return background color
    if( tres.y < 0.0 )
        return worldGetBackground( rd );

    // get position and normal at the intersection point
    vec3 pos = ro + rd * tres.x;
    vec3 nor = worldGetNormal( pos, tres.y );
```

```
// get color for the surface
vec3 scol = worldGetColor( pos, nor, tres.y );

// compute direct lighting
vec3 dcol = worldApplyLighting( pos, nor );

...
```

There is a big difference this time though in applyLighting(). Usually that function tries to be clever and approximate lighting with **soft shadow tricks**, or concavity based ambient occlusion, or just things like blurred shadows maps, actual ambient occlusion, and other techniques. Indeed, that's how realtime demos and games work. However, this time we are not doing any of these (which are too smart for us this time). Instead, our applyLighting() is going to do the simplest (and correct) sampling of lights. Which we can do this in multiple ways. For example, you can pick a random light source (the sky, the sun, one of the lamps in your scene, etc), grab one point in it, and cast one single ray to it. If the ray hits the light source instead of a blocking object, we return some light from the function, otherwise we return black. We can also play differently and actually sample all of the lights, grabbing one random point in it, and casting one shadow ray to that point. It would also be possible to sample the light multiples times and cast a few rays per light. You probably want to do some importance sampling and sample lights differently depending on their size and intensity. But in its simplest form, the function simple casts one shadow ray against the light sources. This will return result in a very noisy image of course, but remember that all of this is run 256 times per pixel anyway (or more), so in practice we are casting many shadow rays per pixel/lens/aperture.

Still this would be a direct lighting renderer. In order to capture indirect lighting, and before we multiply any lighting information with the surface color, we need to cast at least one ray to gather indirect lighting. Again, one could cast a few gather rays, but the idea of a pathtracer is to keep it all simple, and cast only one ray every time (to make one single "light path", therefore the name path-tracing). If the surface we hit is completely diffuse we should just cast our ray in any random direction in the hemisphere centered around the surface normal of the point we are lighting. If the surface was glossy/specular, we should compute the reflected direction of the incoming ray along the surface normal, and cast a ray in a cone centered in that direction (the width of the cone being the glossiness factor of our surface). If the surface was both diffuse and glossy at the same time, then we should choose between both possible outgoing ray directions randomly, with probabilities proportional to the amount of diffuse vs glossiness we wanted for our surface. Once we had our ray, we would start the process again that we already have in place for the direct lighting (cast, calc normal, calc surface color, calc direct lighting and multiply).

This can be done both recursively or iteratively. If it was recursive everything would look like this:

```
vec3 rendererCalculateColor( vec3 ro, vec3 rd, int numLevels )
{
    // after some recursion level, we just don't gather more light
```

```

    if( numLevels==0 ) return vec3(0.0);

    // intersect scene
    vec2 tres = worldIntersect( ro, rd, 1000.0 );

    // if nothing found, return background color
    if( tres.y < 0.0 )
        return worldGetBackground( rd );

    // get position and normal at the intersection point
    vec3 pos = ro + rd * tres.x;
    vec3 nor = worldGetNormal( pos, tres.y );

    // get color for the surface
    vec3 scol = worldGetColor( pos, nor, tres.y );

    // compute direct lighting
    vec3 dcol = worldApplyLighting( pos, nor );

    // compute indirect lighting
    rd = worldGetBRDFRay( pos, nor, rd, tres.y );
    vec3 icol = rendererCalculateColor( pos, rd, numLevels-1 );

    // surface * lighting
    vec3 tcol = scol * (dcol + icol);

    return tcol;
}

```

As said the new function worldGetBRDFRay() returns a new ray direction for the recursive tracer, and again, this can be a random vector in the hemisphere for diffuse surfaces or a ray on a cone around the reflected ray direction based on how glossy vs diffuse the surface is at that point.

The problem with this recursive implementation is that it's not suitable for current generations of graphics hardware (which has no stacks in its shader units). The solution is either to build your own stack if the hardware allows writing to arrays, or go for with an iterative implementation, which is very similar:

```

vec3 rendererCalculateColor( vec3 ro, vec3 rd, int numLevels )
{
    vec3 tcol = vec3(0.0);
    vec3 fcol = vec3(1.0);

    // create numLevels light paths iteratively
    for( int i=0; i < numLevels; i++ )
    {
        // intersect scene
        vec2 tres = worldIntersect( ro, rd, 1000.0 );

        // if nothing found, return background color or break
    }
}

```

```

    if( tres.y < 0.0 )
    {
        if( i==0 )    fcol = worldGetBackground( rd );
        else         break;
    }
    // get position and normal at the intersection point
    vec3 pos = ro + rd * tres.x;
    vec3 nor = worldGetNormal( pos, tres.y );

    // get color for the surface
    vec3 scol = worldGetColor( pos, nor, tres.y );

    // compute direct lighting
    vec3 dcol = worldApplyLighting( pos, nor );

    // prepare ray for indirect lighting gathering
    ro = pos;
    rd = worldGetBRDFRay( pos, nor, rd, tres.y );

    // surface * lighting
    fcol *= scol;
    tcol += fcol * dcol;
}

return tcol;
}

```

In this case we are computing only direct illumination at the hit points and letting the ray bounce literally by changing its origin (to be the surface hit position) and its direction according to the local BRDF, then letting it being casted again. The only trick to keep in mind is that the surface modulation color decreases exponentially as the ray depth increases, for the light hitting a given point in the scene gets attenuated by every surface color that the path hits on its way to the camera. Hence the exponential color/intensity decay $fcol *= scol$;

And that's basically all you need in order to have a basic global illumination renderer able to produce photorealistic images, just a few lines of code and some fast hardware. As I promised, this can take one hour to code. Now, adding extra features as participating media (non uniform density fog), subsurface scattering, efficient hair intersection, etc etc, can take years :) So, choose your feature set carefully before you plan to conquer the world or something like that.

Final notes

I assume that anybody reaching the end of this article knows how to do direct lighting and is able to generate a ray in a random direction with a cosine distribution, a point in a disk or quad and a ray within a cone. The [Total Compendium](#) by Philip Dutr is a good reference. As for reference too, I leave here a couple of the functions used in all of the code and images above too - the one doing the direct lighting computations and the one generating a ray based on the surface BRDF:

```
vec3 worldApplyLighting( in vec3 pos, in vec3 nor )
{
    vec3 dcol = vec3(0.0);

    // sample sun
    {
        vec3 point = 1000.0*sunDirection + 50.0*diskPoint(nor);
        vec3 lray = normalize( point - pos );
        float ndl = max(0.0, dot(lray, nor));
        dcol += ndl * sunColor * worldShadow( pos, lray );
    }

    // sample sky
    {
        vec3 point = 1000.0*cosineDirection(nor);
        vec3 lray = normalize( point - pos );
        dcol += skyColor * worldShadow( pos, lray );
    }

    return dcol;
}
```

The sun is a disk, and the sky is a dome. Note how the sky light doesn't compute the usual diffuse "N dot L" factor. Instead, the code replaces the uniform sampling of the sky dome with a cosine distribution based sampling, which sends more samples in the direction of the normal and less to the sides proportionally to the cosine term, therefore achieving the same effect while casting far less rays (you have probably heard the word "importance sampling" before).

```
vec3 worldGetBRDFRay( in vec3 pos, in vec3 nor, in vec3 eye, in float materialID )
{
    if( random1f() < 0.8 )
    {
        return cosineDirection( nor );
    }
    else
    {
        return coneDirection( reflect(eye,nor), 0.9 );
    }
}
```

In this case the function is 80% diffuse and 20% glossy (with a glossiness cone angle of 0.9 radians).

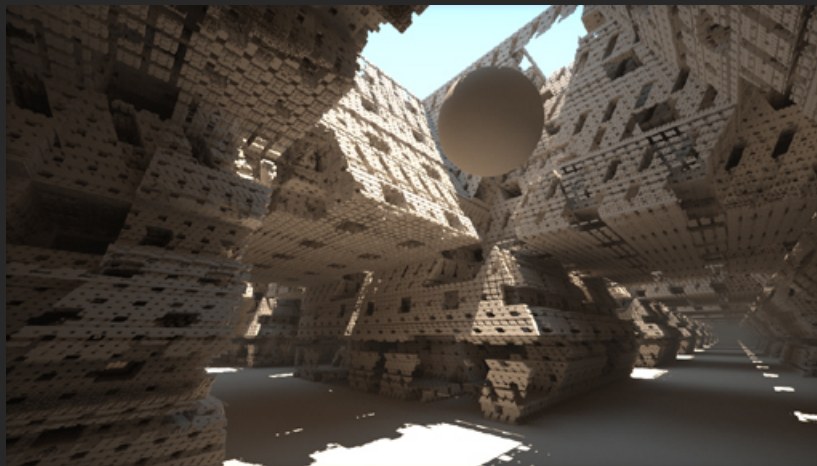
Full source code example

Here are two images with **full source code** that implement the techniques discussed in this article:

<https://www.shadertoy.com/view/Xtt3Wn>:



<https://www.shadertoy.com/view/MsdGzl>:



inigo quilez - learning computer graphics since 1994